Recommender Systems - Course Project

1. What this project is

You will build, evaluate and critique a recommender system on MovieLens 1M, in a team, over six weeks. The project runs in three parts with separate deadlines and is examined in Week 7.

The project is the entire grade for this course. There is no exam. The point is not to build the most accurate model. It is to produce a result you can defend. Every part is graded primarily on the soundness of your method and the honesty of your analysis, and only secondarily on the numbers you obtain. A team that reports "our neural model lost to item-based kNN, and here is why" will score higher than a team reporting a win they cannot account for.

Everything you need is in the lectures. Requirements point to the lecture where the method is taught. If a requirement has no pointer, it is something you are expected to work out yourself.

2. Teams

Teams of 3. Teams of 2 are allowed only if you email at maria.stratigi@tuni.fi before the end of Week 1 with a reason; the scope is not reduced, so this is rarely a good idea. Teams of 4 are not allowed.

Declare your team here: https://docs.google.com/spreadsheets/d/1bF0bhGKrEdFeMLC2RvaLaCWs7z6C9AkPFLeBLrfEWs4/edit?usp=sharing by end of Week 1. Feel free to use the “Creating Groups Forum” available on Moodle for making the groups for the course project. Students without a team by that deadline will be assigned to one. Teams cannot be dissolved after the Part 1 deadline except in case of a documented problem. If a member becomes inactive, tell us early and ask for help. Such situations will be handled case by case. Telling us in Week 6 is too late to fix anything.

Every submission includes a contribution statement: two or three sentences per member describing what they did. This is not decoration; it feeds into individual grading.

3. Dataset: MovieLens 1M

Every team uses MovieLens 1M. Download it yourself from GroupLens (https://grouplens.org/datasets/movielens/1m/) and follow their usage terms. Parsing it is part of the work. The files are plain text with `::` separators. Files:

ratings.dat: ~1,000,209 ratings on a 1–5 scale, with timestamps - user, item, rating, time
movies.dat: ~3,700 movies with title, release year, and genre labels (18 genres, multi-label)
users.dat: ~6,040 users with self-reported gender, age band, occupation and zip code
This is the user–item matrix from Lecture 1.1, slide 6, in real form.

Note: The dataset is dense: roughly 4.5% of the matrix is filled, against the 1% or less described in Lecture 1.1. Timestamps record when a rating was entered, not when the film was watched. The user attributes are self-reported, binary in the case of gender, and from 2003. If you use them, treat them as proxies of limited validity and say so (Lecture 5.2, slide 6). Everyone works on the same data, so your results are directly comparable with every other team's.

4. Tools and languages

Python is strongly recommended for this project and is the language for which the teaching staff can provide the most support. C++ is also acceptable. You may use another programming language, but the teaching staff cannot provide support for language-specific or programming problems in other languages. Regardless of language, the requirements below are mandatory and are about reproducibility, not syntax:

1. The complete project must run from a single documented command. A README.md must state how to obtain the data, how to install all dependencies, and the exact command that reproduces every number and generated figure in your report. For example:
python run_all.py --data-dir /path/to/ml-1m The dataset path must be configurable; do not require the examiner to edit the source code or use paths specific to your own computer.

2. All dependencies required to run the project must be explicitly specified and pinned to appropriate versions (for example, in requirements.txt or environment.yml). Include the required programming-language version as well. The submitted instructions must be sufficient to recreate the environment used for your experiments. Do not assume that a package is available unless you declare it.

3. Use fixed random seeds and report them. Seeds may be hard-coded, provided they are clearly documented and used consistently.

4. All reported metrics must be written using exactly the results template and schema provided on the course page. Do not rename, remove, or reinterpret required fields. The common format is mandatory so that results can be checked and compared consistently across teams.

5. The project must not depend on absolute paths or on intermediate files that exist only on your computer. Generated or intermediate files must either be included where appropriate or, preferably, be recreated automatically by the documented command.

Libraries are encouraged for baselines and infrastructure: RecBole, Implicit, LightFM, Surprise, RecPack, Elliot, scikit-learn, PyTorch, or other appropriate libraries. The self-implementation requirement does not apply to Part 1. In Part 2, however, you must implement at least one core recommendation algorithm yourself from the lecture slides or the original paper, without copying an existing implementation. The self-implemented algorithm must be one of the additional models reported in Part 2 and must be identified clearly.

Reasonable choices:

- User-based or item-based kNN: Lecture 1.2, slides 5–10
- Matrix factorization with biases and regularisation: Lecture 2.1, slides 4, 7–9
- BPR: Lecture 2.1, slide 12
- Content-based profile matching: Lecture 2.2, slides 4–7
- Markov-chain or session-kNN sequential model: Lecture 4.1, slides 6–7
- MMR re-ranking: Lecture 5.1, slide 9

The self-implementation is a hard Part 2 requirement and is checked at the examination. You will be asked how your implementation works. MovieLens 1M fits in memory on any laptop, and ranking the full catalogue of ~3,700 items is fast. Your submission must be runnable using the documented environment and command. If the teaching staff cannot reproduce your results from the submitted instructions, you may be asked, with little notice, to demonstrate that the project runs on your own computer.

Provided: the mandatory results CSV template/schema https://docs.google.com/document/d/19nhPiv6es82L3irhuSqo54b10AaXXAoY/edit?usp=sharing&ouid=103880640392539521217&rtpof=true&sd=true and the mandatory report template https://drive.google.com/file/d/19jE2N5VpbcS0Xwxfw-jIGTTqfU1yqgs7/view?usp=sharing . You need to download these templates for your own use. From File – Download. You must use the provided report template and results template. The same report is developed throughout all three parts: do not create a separate report for each part. Instead, extend and revise the existing report as the project progresses. All explanations, methodological decisions, justifications, analyses, and other report content requested in Parts 1–3 must be included in this report. You

are free to write your own metrics - if you do, show that your implementation agrees with the reference on a sample.

GitHub repository. Each team must create a GitHub repository containing the complete code for the project. Include the repository URL in the report. You are strongly advised to use the same repository throughout Parts 1, 2 and 3, updating it as the project progresses rather than creating a separate repository for each part. The repository must contain the README and all files required to reproduce the submitted results.

5. The three parts

Part 1: Data, protocol, baselines

Due: Sunday, end of Week 3, 15% of the project grade.

Establish the ground you will stand on for the rest of the project. The deadline falls at the end of the week in which Lecture 3.1 covers evaluation, so every protocol decision below is one you can justify from the lectures.

Required:
- The data loaded and characterized: users, items, interactions, density; distribution of interactions per user and per item, with a plot: this is the long tail from Lecture 5.1, slide 5; the rating distribution; the time span and how activity is distributed across it.
- A temporal split into train/validation/test (Lecture 3.1, slide 4). Document exactly where you cut and why. If you filter out low-activity users or items, state the threshold and how many rows it removed.
- Two baselines:
    1. **Item-based kNN** (Lecture 1.2, slides 5–10). Tune the neighborhood size k and the similarity measure.
    2. **Most popular**. There is nothing to tune here, but there are choices: all-time popularity or a recent window; count all ratings or only positive ones. State which choices you made, and report at least two variants.
- Metrics: Precision@10, Recall@10, NDCG@10 (Lecture 3.1, slides 7–8), plus catalogue coverage (Lecture 3.1, slide 9). Ranked against the full catalogue.
- Include in the report a justification for each protocol decision: the split, the feedback treatment, the filtering, the candidate set, and the choice of k. One or two sentences each are sufficient, referring to the relevant lecture material.

Submit: GitHub repository link, (ii) results using the provided results template, (iii) the current version of the report using the provided report template (maximum 4 pages).
This is the first version of the single report that you will extend in Parts 2 and 3.

What separates good from excellent: noticing something non-obvious about data that everyone else is also looking at. How extreme is the long tail here, exactly? What does the burstiness of the timestamps do to your temporal split? Did your filtering threshold change the results by more than the gap between your two baselines?

Part 2: Models and honest comparison

Due: Sunday, end of Week 5, 30% of the project grade

Required: at least three models beyond the Part 1 baselines, drawn from at least two different paradigms:

- Neighbourhood CF: user-based kNN; item-based kNN with a different similarity or normalization than Part 1, Lecture 1.2, slides 5–10
- Latent-factor CF: matrix factorization with biases and regularization; the same model fitted with ALS instead of SGD, Lecture 2.1, slides 4, 7–9

- Ranking-loss CF: BPR or another pairwise objective, Lecture 2.1, slides 11–12
- Content-based: profile matching over genre and release-year metadata, Lecture 2.2, slides 4–7
- Hybrid: weighted, switching, cascade, feature combination, meta-level, Lecture 2.2, slide 10
- Neural: autoencoder or Mult-VAE; two-tower, Lecture 3.2, slides 7, 9–10
- Sequential: Markov chains; session-kNN; GRU4Rec; SASRec. Lecture 4.1, slides 6, 8–9

Two models from the same category count as one paradigm. One of the additional Part 2 models must be your self-implemented algorithm. This requirement begins in Part 2: neither of the Part 1 baselines satisfies the self-implementation requirement.

Also required:

1. Hyperparameter tuning on the validation set only, with the search space and budget reported. Your baselines get the same budget as your own favorite model. Lecture 3.1, slide 10 explains why this is the single most common source of false gains.
2. Variability reported: multiple seeds with confidence intervals, or a paired statistical test across users (Lecture 3.1, slide 10). A difference reported without an interval will not be accepted as a difference.
3. A disaggregated result table: overall metrics plus results broken down by at least two of - items with few vs many training interactions, low- vs high-activity users, popular vs long-tail items, early vs late test period (Lecture 3.1, slide 9; Lecture 5.2, slide 6).
4. A sparsity ablation. Subsample the training interactions to 50% and 20% and report how each model's performance, and, more interestingly, the ranking between models, changes. A model that wins on the full data and loses at 20% is a finding worth reporting (Lecture 1.2, slide 12).
5. Announce your Part 3 track with this submission.

Submit: (i) GitHub repository link, (ii) updated results using the provided results template, (iii) the updated version of the same report submitted in Part 1 (maximum 8 pages).
Extend the report to include all explanations, justifications, experimental details and analyses required in Part 2.

What separates good from excellent: the disaggregation and the ablation doing real work. Anyone can produce a table of overall NDCG. Excellent teams find that their headline winner loses on cold items or collapses under sparsity and say so plainly instead of burying it.

Part 3: Beyond accuracy

Code, results and presentation slides due: Sunday, end of Week 6.
Final report due: Sunday, end of Week 7. 35% of the project grade

The motivation is Lecture 1.1, slide 12 ("Accuracy is not the goal"). Choose **one** track, implement an intervention, and measure what it costs.

Track A: Diversity and popularity bias (Lecture 5.1)

Use your best-performing model from Part 2 as the base recommender.

1. Implement one re-ranking intervention:
    - MMR (Lecture 5.1, slide 9)
    - calibration-based re-ranking (slide 11)
    - an exposure-aware re-ranker (slide 12)
2. Use genre vectors as the item representation.
3. Sweep the intervention's trade-off parameter and plot the accuracy–diversity frontier (slide 13).
4. Report intra-list diversity, catalogue coverage, Gini and calibration error (slide 7).

5. State exactly which item-distance measure you used.

Track B: Fairness (Lecture 5.2)

MovieLens records no studio, publisher or rights-holder, so it has no provider structure.

1. Construct a provider proxy from release decade.
    a. Group items into release-decade categories such as 1930s–1970s, 1980s, 1990s and 2000s
    b. Adjust the boundaries if appropriate for your data and report the resulting group sizes.
2. Measure position-weighted exposure for each provider-proxy group and compare it with that group's share of the catalogue (Lecture 5.2, slide 8).
3. Measure per-group consumer-side accuracy using the user attributes (slide 6).
4. Choose and clearly state the fairness notion you adopt and justify it (slides 9–10).
5. Apply one fairness mitigation at one of the three stages discussed in slide 12.
6. Vary the mitigation strength and plot the resulting fairness–performance frontier.

Track C: Group recommendations (Lecture 4.2)

1. Choose one synthetic group type (Lecture 4.2, slide 12): random, similar or divergent.
2. Choose the group size (3-5) and justify your choice.
3. Implement at least three aggregation strategies (slide 5).
4. For each aggregation strategy, evaluate:
    a. group satisfaction
    b. disagreement
    c. minimum member satisfaction (slides 8–9).
5. Choose one of the aggregation strategies and apply it to a sequential group-recommendation scenario (slide 10).
6. Run a sequence of recommendation rounds.
7. Carry satisfaction forward between rounds.
8. Analyze how member and group satisfaction develop over the sequence.

Required in all tracks:

Complete all the following:

1. Implement the intervention or extension required by your chosen track and verify that it works.
2. Characterize the central trade-off in your track.
    a. Tracks A and B: vary the relevant parameter and report the trade-off as a curve, not a single point (Lecture 5.1, slide 13).
    b. Track C: compare how the aggregation strategies balance group satisfaction, disagreement and minimum member satisfaction, and analyse how satisfaction changes in the sequential scenario.
3. State explicitly what is gained and what is lost by the intervention or strategy, and who bears the cost where applicable (Lecture 5.2, slide 13).
4. Include a "one number we cannot explain" section in the report. Identify one result that you do not fully understand and describe what you would do to investigate it. This section is graded, and an honest one scores well.

Submit by the end of Week 6: (i) GitHub repository link, (ii) final results using the provided results template, (iii) presentation slides.

Submit by the end of Week 7: (i) the final version of the same report developed in Parts 1 and 2, maximum 12 pages excluding references, (ii) the final report must contain the explanations, justifications, experimental details and analyses required across all three parts.

What separates good from excellent: treating the trade-off as the finding. For Tracks A and B, excellent teams can explain the shape of the frontier: where it is flat, where it turns, and what operating point they would choose and why. For Track C, excellent teams can explain how the aggregation strategies balance the different group objectives and what changes when recommendations are made sequentially.

6. Week 7: Examination

Two sessions of two hours, both compulsory for all teams. Ten minutes per team: eight minutes of presentation and two minutes of questions. Exact schedule on the course page. Present the work you submitted at the end of Week 6. Cover what you built, your headline result, and the single most interesting thing you found, including, if it applies, the thing that did not work. Everyone used the same data, so do not spend time describing MovieLens.

Every team member presents part of the talk, and every member must answer at least one question. Questions may concern parts of the work you did not personally lead, so make sure the whole team understands the whole project.

Attendance at both sessions is mandatory and individual. If your team presents but a member is absent, the team is examined normally; the absent student is rejected.

7. Peer evaluation

Each team is assigned **three other teams to evaluate**, using the presentations in Week 7. Assignments are published in Week 6. The teams can make questions during the presentations. For each assigned team you submit a short, structured evaluation: (i) A score from 1 to 5 on each of the four rubric criteria in Section 8 (next): methodological soundness, depth of analysis, implementation quality, communication with one sentence of justification per criterion. (ii) The single strongest element of their work, and the single weakest, in one sentence each.
You are graded on the quality of your evaluations, whether your scores are justified, specific and consistent with the evidence, and not on whether they match ours.

Due: Sunday, end of Week 7, together with your final report. Your evaluations do not determine other teams' grades; we may use them as a cross-check when our own assessments are borderline.
This component is mandatory. Not submitting it or submitting three evaluations that consist of numbers without justification, means failing the course.

8. Grading

The project is 100% of the course grade.
Part 1: 15%
Part 2: 30%
Part 3: 35%
Week 7 examination: 10%
Peer evaluation: 10%

Rubric applied to each written part:

- Methodological soundness: 35%, Evaluation setup correct and specified; baselines tuned as hard as the proposed model; claims supported by intervals

- Depth of analysis: 30%, Results disaggregated and interpreted; failure cases examined; the team knows why their numbers came out that way
- Implementation quality: 20%, Reproducible from the README; the self-implemented algorithm is correct; code is readable
- Communication: 15%, Report is within the page limit, precise, and free of unsupported claims; figures are labelled and readable

Note that a well-executed project reporting that nothing beat the most-popular baseline can receive full marks. A project reporting a large improvement obtained by a flawed protocol cannot.

Individual grades. Each student's grade is the team grade adjusted by a factor between 0.8 and 1.1, determined by the contribution statements and your performance in the Week 7 questioning. Most students receive 1.0. The mechanism exists so that unequal effort has consequences in both directions.

Passing the course

Attendance at both Week 7 sessions is individually mandatory. A student who does not appear is rejected from the course. You must score at least 40% of the maximum on every one of the five components: Part 1, Part 2, Part 3, the Week 7 examination, and the peer evaluation. Falling below 40% on any single component means failing the course, regardless of your total.

9. Rules and practicalities

Deadlines are Sundays at 23:59, for every part. Parts cannot be revised and submitted again after grading.

Late submissions. A Part submitted up to one week late is accepted with a 30% deduction of that part's score. After one week it is not accepted at all. There is no further extension.

Documented illness or comparable circumstances are handled separately. Contact the instructor before the deadline where possible.

Use of AI assistants is permitted and must be disclosed in a short section of the report: which tools were used, and for what. You are fully responsible for everything you submit, and at the examination you must be able to explain any line of your code. Code you cannot explain will be treated as not your own.

Getting help. Weekly project support hours are held **every Wednesday from 8:00 to 10:00 in TAU Konetalo K1241**. Use them early. Most of what goes wrong in this project goes wrong in the first weeks and stays invisible until the end.

10. Quick calendar

All deadlines are Sunday at 23:59.
End of Week 1: Team registration
End of Week 3: Part 1 - data, protocol, baselines
End of Week 5: Part 2 - models, comparison, sparsity ablation; Part 3 track announced
End of Week 6: Part 3 - code, results and presentation slides; peer-evaluation assignments published
Week 7: Examination: two sessions, 10 minutes per team - attendance mandatory
End of week 7: Part 3 - final report, and your three peer evaluations